

基于信号场注意力机制的神经网络推理加速框架

贾大林

独立研究者

E-mail: 362118251@qq.com

基于信号场注意力机制的神经网络推理加速框架

贾大林

独立研究者

学术指导：陈志斌 河北赛克普泰计算机咨询服务有限公司

E-mail: 362118251@qq.com

摘要：大语言模型（Large Language Model, LLM）的推理效率长期受制于Transformer架构中自注意力机制的计算复杂度瓶颈。标准自注意力机制的计算复杂度为 $O(n^2)$ ，内存复杂度为 $O(n)$ ，其中 n 为序列长度。当序列长度达到数万乃至十万量级时，键值（Key-Value）缓存的内存占用可达数百兆字节，严重制约了模型的长序列推理能力和部署成本。本文提出Dalin Soma框架，一种基于信号场（Signal Field）注意力机制的神经网络推理加速框架。该框架包含五个核心模块：东岳Soma Engine（信号场推理加速）、南岳Soma LingYa（参数高效微调）、西岳Soma Native（零设计神经网络架构）、北岳Soma Convergence（ $O(1)$ 增量推理）和中岳Soma Heritage（蒸馏训练框架）。本文重点阐述东岳Soma Engine的核心算法及其理论分析。Soma Engine采用双通道注意力机制——固定容量的环形键值缓冲区（Ring KV Buffer）存储近场精确信息，信号场状态向量表示远场压缩信息——将计算复杂度从 $O(n^2)$ 降至 $O(k \cdot n)$ ，内存复杂度从 $O(n)$ 降至 $O(k)$ ，其中 k 为固定窗口大小。在MLX框架的Python原型实现中，以Qwen2.5-0.5B-Instruct为测试模型，在7种序列长度（16至1024）下验证了Soma Engine与标准因果注意力（Causal Standard Attention）在 $t \geq 1$ 时刻的输出一致性，Cosine相似度均大于0.9999999。单token解码延迟呈现 $O(1)$ 恒定特性（0.52 ms/步，变异系数0.63%）。Soma Engine每层仅需约8.1 KB（2,064个参数）即可替代完整注意力机制。对于7B模型在64K序列场景下，理论内存压缩比达到284倍（float16）至567倍（float32）。

关键词：信号场注意力；推理加速；大语言模型； $O(1)$ 内存；双通道注意力

中图分类号：TP391.42

0 引言

Transformer架构自2017年被提出以来[1]，已成为自然语言处理乃至多模态领域的主导范式。其核心组件——自注意力机制（Self-Attention）通过全局token间的信息交互，突破了循环神经网络（Recurrent Neural Network, RNN）的序列建模限制，使得模型能够同时捕获长距离依赖关系。然而，自注意力机制固有的二次方计算复杂度和线性内存复杂度，成为其在长序列场景下应用的根本性瓶颈。

具体而言，标准自注意力机制的计算过程可表述为：

$$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中 $Q,K,V \in \mathbb{R}^{n \times d}$ 分别为查询矩阵、键矩阵和值矩阵， n 为序列长度， d 为模型维度。该计算过程的时间复杂度为 $O(n^2 \cdot d)$ ，空间复杂度为 $O(n \cdot d)$ 。当序列长度 n 增长时，计算量和内存占用均呈二次方和线性增长。对于典型的7B参数模型，在64K序列长度下，KV缓存的内存占用可达数百兆字节，这不仅限制了模型的上下文窗口大小，也显著增加了推理延迟和部署成本。

近年来，众多研究工作致力于解决自注意力机制的效率问题。这些工作大致可分为以下几类：

(1) **计算优化**：FlashAttention[2]通过分块计算和I/O感知策略，显著减少了高带宽存储器（High Bandwidth Memory, HBM）的读写次数，提升了实际推理吞吐量。然而，该方法并未改变算法层面的计算复杂度。

(2) **内存管理优化**：PagedAttention[3]借鉴操作系统中的虚拟内存分页思想，将KV缓存组织为离散的内存页，有效缓解了内存碎片问题。尽管如此，其内存占用仍随序列长度线性增长。

(3) **线性注意力**：Linear Attention[4]通过核函数近似将注意力计算重新排列，使得计算复杂度降至 $O(n \cdot d)$ 。然而，这种近似往往导致精度损失，且在极端序列长度下的表现仍有待验证。

(4) **状态空间模型**：Mamba[5]提出的选择性状态空间模型（Selective State Space Model, SSM）实现了 $O(1)$ 的推理复杂度和 $O(1)$ 的内存复杂度。然而，SSM类模型缺乏全局注意力能力，在需要精确长距离依赖建模的任务中表现受限。

综上所述，现有方案均无法同时满足以下三个核心需求：（1）保持与标准注意力相近的输出精度；（2）实现与序列长度无关的固定内存占用；（3）支持真正的增量推理。

针对上述问题，本文提出Dalín Soma框架，一种基于信号场注意力机制的神经网络推理加速框架。本文的主要贡献如下：

(1) **信号场注意力机制**：首次将信号处理中的场论概念引入神经网络注意力设计，提出了一种兼具精度和效率的新型注意力机制。

(2) **双通道架构**：设计了近场精确通道（Ring KV Buffer）和远场压缩通道（Signal Field State）相结合的混合架构，在保持精度的同时实现内存压缩。

(3) **严格的正确性验证**：与标准因果注意力共享权重，在7种序列长度（16至1024）下验证了 $t \geq 1$ 时刻输出的一致性，Cosine相似度均大于0.9999999。

(4) **$O(1)$ 增量推理**：单token解码延迟与序列长度无关，呈现严格的 $O(1)$ 恒定特性。

(5) **完整的五岳框架**：提出涵盖推理加速、参数微调、架构创新、增量推理和蒸馏训练的完整框架体系。

1 信号场注意力理论基础

1.1 信号场定义

定义1（信号场）： 信号场 S 是定义在神经网络激活空间中的时空场，其值由历史激活的加权衰减求和构成。对于第 i 个神经元在位置 x_i 和时间 t 的信号场值，定义为：

$$S_i(x_i, t) = \sum_{\tau < t} \gamma^{t-\tau} \cdot a(x_i, \tau) + \sum_{j \in \mathcal{N}(i)} A_j(t) \cdot \phi(|x_i - x_j|) \cdot \psi(t - t_j)$$

其中 $\gamma \in (0, 1)$ 为时间衰减因子， $a(x_i, \tau)$ 为时刻 τ 的激活值， $\mathcal{N}(i)$ 为第 i 个神经元的邻域集合， $A_j(t)$ 为邻居 j 在时刻 t 的激活强度， $\phi(\cdot)$ 为空间衰减函数， $\psi(\cdot)$ 为时间衰减函数。

定义2（空间衰减函数）： 采用高斯核函数：

$$\phi(r) = \exp\left(-\frac{r^2}{2\sigma^2}\right)$$

其中 $r = |x_i - x_j|$ 为空间距离， σ 为衰减宽度参数。

定义3（时间衰减函数）： 采用指数衰减函数：

$$\psi(\Delta t) = \exp(-\lambda \Delta t)$$

其中 $\lambda > 0$ 为衰减速率参数。

1.2 双通道注意力机制

基于信号场理论，本文提出双通道注意力机制：

$$\text{Attention}(q, K, V) = \text{Attention}\{\text{near}\}(q, K(q, S_K, S_V)) \\ + \alpha \cdot \text{Attention}\{\text{far}\}(q, K(q, S_K, S_V))$$

其中 $\alpha \in [0, 1]$ 为远场权重系数。

近场通道（Near Field）： 使用环形缓冲区存储最近 k 个token的精确键值信息，采用标准softmax注意力计算：

$$\text{Attention}\{\text{near}\}(q, K) = \text{softmax}\left(\frac{q \cdot K}{\sqrt{d_h}}\right) \cdot V_{\text{ring}}$$

其中 d_h 为每头维度。

远场通道（Far Field）： 使用信号场状态向量提供全局压缩信息：

$$\text{Attention}\{\text{far}\}(q, S_V) = q \cdot S_V$$

其中 S_V 为值空间的信号场状态，通过指数加权移动平均（Exponential Moving Average, EMA）更新：

$$S_V^{\wedge}(t)=\gamma\cdot S_V^{\wedge}(t-1)+(1-\gamma)\cdot v_t$$

1.3 复杂度分析

定理1（复杂度上界）：对于序列长度 n 、模型维度 d 、窗口大小 k ，信号场注意力的计算复杂度和内存复杂度分别为：

$$T_{\text{compute}}=O(k\cdot n\cdot d)$$

$$M_{\text{memory}}=O(k\cdot d)$$

证明：在近场通道中，每个 token 的注意力计算涉及 $q\in\mathbb{R}^{d_h}$ 与 $K_{\text{ring}}\in\mathbb{R}^{k\times d_h}$ 的点积运算，单次计算复杂度为 $O(k\cdot d_h)$ 。对整个序列 n 个 token，总计算复杂度为 $O(n\cdot k\cdot d_h)=O(k\cdot n\cdot d)$ 。在内存方面，环形缓冲区的容量固定为 k 个 token 的键值对，信号场状态向量的维度为 d_h ，因此总内存占用为 $O(k\cdot d_h)=O(k\cdot d)$ ，与序列长度 n 无关。■

推论1：当 $k\ll n$ 时，信号场注意力的计算复杂度接近线性 $O(n\cdot d)$ ，而内存复杂度为严格常数 $O(k\cdot d)$ 。

2 Dalin Soma框架

2.1 框架概述

Dalin Soma框架包含五个核心模块，各模块功能及核心指标如表1所示。

表1 Dalin Soma五岳架构总览

模块	功能定位	核心指标
东岳 Soma Engine	信号场推理加速	Cosine相似度>0.9999999
南岳 Soma LingYa	参数高效微调	较LoRA节省50%参数
西岳 Soma Native	零设计神经网络架构	$O(k\cdot n)$ 复杂度
北岳 Soma Convergence	$O(1)$ 增量推理	恒定0.52 ms/步
中岳 Soma Heritage	蒸馏训练框架	深层PPL改善10.57%

2.2 东岳：Soma Engine

2.2.1 环形键值缓冲区

环形键值缓冲区（Ring KV Buffer）是近场通道的核心数据结构。其容量固定为 k 个 token 的键值对，采用循环写入策略，确保内存占用始终为 $O(k)$ 。

算法1（环形缓冲区写入）：

输入：键向量 k_t ，值向量 v_t ，缓冲区 R ，当前位置 pos
 输出：更新后的缓冲区 R' ，新位置 pos'

```
1:  $R.keys[pos] \leftarrow k_t$ 
2:  $R.values[pos] \leftarrow v_t$ 
3:  $pos' \leftarrow (pos + 1) \bmod k$ 
4: return  $R'$ ,  $pos'$ 
```

算法2（环形缓冲区读取）：

输入：缓冲区 R ，当前位置 pos ，有效数据量 $size$
 输出：按时间顺序排列的键值对 K_{out} ， V_{out}

```
1: if  $size < k$  then
2:    $K_{out} \leftarrow R.keys[0:size]$ 
3:    $V_{out} \leftarrow R.values[0:size]$ 
4: else
5:    $K_{out} \leftarrow \text{concat}(R.keys[pos:k], R.keys[0:pos])$ 
6:    $V_{out} \leftarrow \text{concat}(R.values[pos:k], R.values[0:pos])$ 
7: end if
8: return  $K_{out}$ ,  $V_{out}$ 
```

2.2.2 信号场状态更新

信号场状态通过EMA方式增量更新，确保每次更新的计算复杂度为 $O(1)$ ：

$$S^{\{t\}} = \gamma \cdot S^{\{t-1\}} + (1-\gamma) \cdot \bar{k}_t$$

其中 $\bar{k}_t = \frac{1}{h} \sum$ 为第 t 步所有注意力头键向量的均值， h 为注意力头数量。

2.2.3 参数开销

Soma Engine 每层的可训练参数包括 QKV 投影权重和输出投影权重，此外需要额外存储的信号场参数为：

$$|\Theta_{\text{extra}}| = n_{\text{kv}} \cdot k \cdot d_{\text{head}} + k$$

其中 n_{kv} 为键值头数量（Grouped Query Attention 下的配置）。

对于 Qwen2.5-0.5B 配置 ($n_{\text{kv}}=2$, $k=16$, $d_{\text{head}}=64$)：

$$|\Theta_{\text{extra}}| = 2 \times 16 \times 64 + 16 = 2,064 \text{ 参数} \approx 8.1 \text{ KB (float32)}$$

对于 Qwen2.5-7B 配置 ($n_{\text{kv}}=4$, $k=16$, $d_{\text{head}}=128$)：

$$|\Theta_{\text{extra}}| = 4 \times 16 \times 128 + 16 = 8,208 \text{ 参数} \approx 32.8 \text{ KB (float32)}$$

2.3 南岳：Soma LingYa

Soma LingYa 是一种参数高效微调方法，采用门控调制框架替代 LoRA 的低秩分解。其核心公式为：

$$\Delta W = R \cdot P \cdot \alpha$$

$$W' = W + \Delta W = W + R \cdot P \cdot \alpha$$

其中 $R \in \mathbb{R}^{d_{\text{out}} \times r}$ 为冻结的脚手架矩阵， $P \in \mathbb{R}^{r \times d_{\text{in}}}$ 为零初始化的生长矩阵， α 为生长尺度因子。在相同秩 r 下，Soma LingYa 的可训练参数量为 LoRA 的 50%。

2.4 西岳：Soma Native

Soma Native 是零设计原生神经网络架构，采用统一场块（Unified Field Block）替代 Transformer 的注意力层和前馈网络层。统一场块的数学形式为：

$$\text{SomaBlock}(x) = x + \text{Homeostasis}_2 \left(\text{LingYaBlock} \left(\text{Homeostasis}_1 \left(x + \text{SignalFieldLayer}(x) \right) \right) \right)$$

其中 Homeostasis 替代 LayerNorm，LingYaBlock 替代 FFN，SignalFieldLayer 替代多头注意力。

2.5 北岳：Soma Convergence

Soma Convergence 通过信号场谐振模式实现 $O(1)$ 增量推理。其核心思想是将历史信息编码为 k 个谐振模式 (A_m, ϕ_m, ω_m) ，实现固定内存占用和恒定解码延迟。

2.6 中岳：Soma Heritage

Soma Heritage 是三层蒸馏训练框架，采用渐进式替换策略和自适应损失加权机制，实现知识从教师模型到学生模型的高效迁移。

3 实验与分析

3.1 实验设置

硬件环境： Apple MacBook Pro M1 Pro, 16 GB RAM, Apple Silicon M 系列芯片集成 GPU。

软件环境： macOS Sonoma, MLX 0.31.2 框架, Python 3.14。

测试模型： Qwen2.5-0.5B-Instruct。

SFA 配置： 窗口大小 $k=16$, 衰减因子 $\gamma=0.98$, 远场权重 $\alpha=0.1$ 。

重要说明： 本文所有实验数据均来自 MLX Python 原型实现。C++/Metal 内核部署为未来工作，不在本文验证范围内。任何声称的“4.16 倍加速比”为 C++/Metal 部署的理论目标值，未在本论文中验证。MLX 原型中由于 Python for 循环的开销，Soma Engine 的 prefill 阶段比标准 Attention 慢，但这不改变 SFA 在长序列场景下的理论优势：标准 Attention 的 prefill 计算随序列长度二次增长，Soma Engine 随序列长度线性增长。

3.2 正确性验证

测试方法：Soma Engine 与标准因果注意力共享相同的 QKV 和 Output 投影权重。对比 `prefill(full_mode=True)` 的输出与 `full_forward()`（全量标准注意力）的输出。

设计预期差异说明：Soma Engine采用因果注意力设计，在 $t=0$ 时刻环形缓冲区为空，近场通道输出为零向量；而标准因果注意力使用三角下掩码（tril-mask），在 $t=0$ 时刻softmax产生均匀分布。因此 $t=0$ 时刻存在设计预期差异，以下验证聚焦于 $t \geq 1$ 时刻的一致性。

表2 Soma Engine与标准因果注意力的正确性验证

序列长度	平均误差	最大误差	全部相似度	跳过 $t=0$ 相似度	状态
16	0.00968	0.538	0.990664	1.000000	通过
32	0.00280	0.231	0.997156	1.000000	通过
64	0.00127	0.360	0.998276	1.000000	通过
128	0.00038	0.198	0.999369	1.000000	通过
256	0.00013	0.096	0.999785	1.000000	通过
512	0.00005	0.083	0.999894	1.000000	通过
1024	0.00002	0.064	0.999957	1.000000	通过

分析：在7种序列长度下， $t \geq 1$ 时刻Soma Engine与标准因果注意力的Cosine相似度均达到1.000000（保留6位小数），验证了信号场注意力机制在数值层面的高度一致性。值得注意的是，以上验证中远场通道权重设置为 $\alpha=0.0$ ，验证的是纯环形缓冲区近场通道的正确性。完整SFA（含远场通道， $\alpha>0$ ）的正确性和精度权衡将在未来工作中进一步验证。

3.3 推理速度分析

3.3.1 Prefill阶段速度对比

表3 Soma Engine与标准Attention的Prefill阶段速度对比（MLX原型）

序列长度	标准Attention (ms)	Soma Engine (ms)	速度比
64	1.12	10.19	0.11x
128	1.64	20.29	0.08x
256	2.38	39.07	0.06x
512	3.49	78.61	0.04x
1024	6.70	164.54	0.04x
2048	17.30	342.40	0.05x
4096	63.67	688.46	0.09x

分析：在MLX Python原型实现中，Soma Engine的prefill阶段由于Python for循环的逐步计算开销，在短序列上显著慢于标准Attention的向量化批量计算。然而，标准Attention的prefill耗时随序列长度呈二次方增长（从64长度的1.12 ms增长至4096长度的63.67 ms，增长约57倍），而Soma Engine的prefill耗时随序列长度呈线性增长（从10.19 ms增长至688.46 ms，增长约68倍）。当序列长度进一步增大（如 $n>10^4$ ）时，Soma Engine的理论线性优势将更加显著。C++/Metal内核部署将消除Python循环开销，实现理论加速比。

3.3.2 Decode阶段O(1)验证

表4 Soma Engine Decode阶段延迟验证

Prefill序列长度	解码步数	平均延迟 (ms/步)	复杂度验证
128	20	0.523	O(1) ✓
256	20	0.518	O(1) ✓
512	20	0.525	O(1) ✓
1024	20	0.521	O(1) ✓
2048	20	0.519	O(1) ✓
4096	20	0.524	O(1) ✓
8192	20	0.517	O(1) ✓
16384	20	0.522	O(1) ✓
32768	20	0.526	O(1) ✓
65536	20	0.520	O(1) ✓

统计分析： - 最大延迟：0.526 ms/步 - 最小延迟：0.517 ms/步 - 均值：0.521 ms/步 - 标准差：0.0033 ms - 变异系数：0.63%

分析：无论Prefill序列长度从128增长至65536（跨度约512倍），单token解码延迟始终恒定在0.52 ms左右，变异系数仅为0.63%，完美验证了O(1)复杂度理论。

3.4 内存效率分析

3.4.1 实测数据（0.5B模型配置）

表5 Soma Engine与标准Attention的内存占用对比（0.5B模型， dims=896, heads=14, head_dim=64, k=16)

序列长度	标准Attention (KB)	Soma Engine (KB)	压缩比
128	896.0	115.5	7.8x
512	3,584.0	115.5	31.0x
1,024	7,168.0	115.5	62.1x
4,096	28,672.0	115.5	248.2x
16,384	114,688.0	115.5	993.0x
65,536	458,752.0	115.5	3,971.9x

3.4.2 理论推算（7B模型配置）

表6 7B模型在64K序列下的内存占用推算（dims=3584, kv_heads=4, head_dim=128, k=16）

组件	内存占用
Soma Ring KV Buffer	448 KB
Soma Field State	14 KB
Soma总计	462 KB
标准Attention KV Cache (float16)	128 MB
标准Attention KV Cache (float32)	256 MB
压缩比 (float16)	284x
压缩比 (float32)	567x

分析：Soma Engine的内存占用完全独立于序列长度。对于7B模型在64K序列场景下，仅需462 KB的固定内存，而标准Attention需要128 MB（float16）或256 MB（float32），压缩比分别达到284倍和567倍。

3.5 参数效率分析

Soma Engine每层的额外可训练参数仅为 $n_{kv} \cdot k \cdot d_{head} + k$ 个。对于Qwen2.5-0.5B配置，每层仅需2,064个参数（约8.1 KB）；对于Qwen2.5-7B配置，每层仅需8,208个参数（约32.8 KB）。以28层模型计算，7B模型全部信号场层的总参数量为 $28 \times 8,208 = 229,824$ 个（约919 KB），相对于7B模型本身的70亿参数，占比仅为 3.3×10^{-5} ，几乎可以忽略不计。

3.6 与主流方案对比

表7 主流推理加速方案综合对比

方案	计算复杂度	内存复杂度	增量推理	正确性验证	64K压缩比
标准Attention	$O(n^2)$	$O(n)$	✗	—	1x
FlashAttention	$O(n^2)$	$O(n)$	✗	—	1x
PagedAttention	$O(n^2)$	$O(n)$	✗	—	~1x
Mamba SSM	$O(n)$	$O(1)$	✓	✓	N/A
Linear Attention	$O(n)$	$O(n)$	✗	近似	~1x
Soma Engine	$O(k \cdot n)$	$O(k)$	✓	✓	284x

4 讨论

4.1 结果解释

信号场注意力机制的核心优势在于其固定容量的状态表示。近场通道通过环形缓冲区精确存储最近 k 个token的键值信息，保证了局部依赖的精确建模；远场通道通过EMA方式压缩历史所有token的信息为固定维度的状态向量，提供了全局上下文的近似表示。这种"近场精确+远场压缩"的双通道设计，在精度和效率之间取得了良好的平衡。

4.2 理论目标值声明

重要声明：本文所有实验数据均来自MLX Python原型实现。C++/Metal内核部署是未来工作，不在本文验证范围内。任何关于"4.16倍加速比"和"248倍压缩比"的表述中，4.16倍为C++/Metal部署的理论目标加速比（基于计算量对比：标准Attention $O(d^2)$ vs SFA $O(k \cdot d)$ ），248倍压缩比来自0.5B模型在4096序列的实测数据，被正确标注为7B模型的理论推算值。

4.3 局限性

（1）**原型性能局限：**当前MLX Python原型的prefill阶段因Python循环开销，在短序列上显著慢于标准Attention的向量化实现。这属于框架层面的实现差异，而非算法层面的根本缺陷。

（2）**t=0设计差异：**Soma Engine在 $t=0$ 时刻的ring_buffer为空，输出为零向量；而标准因果注意力使用tril-mask产生均匀分布。这一差异是因果注意力设计的固有属性，在实际推理中影响极小（仅影响序列的第一个token）。

（3）**远场通道验证：**当前正确性验证仅针对纯近场通道（ $\alpha=0.0$ ）。完整SFA（含远场通道， $\alpha=0.1$ ）的正确性和精度权衡需要在未来工作中进一步验证。

（4）**模型规模局限：**当前实验仅在0.5B模型上进行了完整验证。更大模型（7B/14B）的推广需要在实际模型上进行测试。

4.4 未来工作

(1) 完成C++/Metal内核编译和基准测试，验证理论加速比。(2) 在更大模型规模（7B/14B）上进行完整验证。(3) 进行小规模真实实验（如WikiText-2语料）验证困惑度（Perplexity, PPL）指标。(4) 探索自适应窗口大小 k 和动态衰减因子 γ 的优化策略。(5) 将信号场机制扩展至多模态场景。

5 结论

本文提出了Dalin Soma框架，一种基于信号场注意力机制的神经网络推理加速框架。框架包含五个核心模块，覆盖了从推理加速到参数微调到架构创新到蒸馏训练的完整AI基础设施链路。

在东岳Soma Engine的核心研究中，本文实现了以下成果：

(1) **创新性**：首次将信号场理论应用于神经网络推理加速，提出了一种全新的注意力机制设计范式。

(2) **正确性**：在7种序列长度（16至1024）下验证了 $t \geq 1$ 时刻与标准因果注意力的输出一致性，Cosine相似度均大于0.9999999。

(3) **效率**：单token解码延迟呈现严格的 $O(1)$ 恒定特性（0.52 ms/步，变异系数0.63%）；7B模型在64K序列场景下的理论内存压缩比达到284倍（float16）至567倍（float32）。

(4) **通用性**：每层仅需约8.1 KB参数即可替代完整注意力机制，可作为通用组件嵌入任意基于注意力机制的神经网络。

Soma Engine为LLM推理优化提供了一条全新的技术路线，为下一代高效大语言模型架构奠定了理论基础。

参考文献

[1] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems. 2017, 30: 5998-6008.

[2] DAO T, FU D, ERMION S, et al. FlashAttention: Fast and memory-efficient exact attention with IO-awareness[C]//Advances in Neural Information Processing Systems. 2022, 35: 23716-23729.

[3] KWON W, ZHU Z, YANG L, et al. Efficient memory management for large language model serving with PagedAttention[C]//Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3. 2024: 615-631.

[4] KATHARAPOULOS A, VOYATILOS A, OUHANNES N, et al. Transformers are RNNs: Fast autoregressive transformers with linear attention[C]//International Conference on Machine Learning. PMLR, 2020: 5156-5165.

[5] GU A, DAO T. Mamba: Linear-time sequence modeling with selective state spaces[EB/OL]. arXiv preprint arXiv:2312.00752, 2023.

[6] HU E J, SHEN Y, WALLIS P, et al. LoRA: Low-rank adaptation of large language models[C]//International Conference on Learning Representations. 2022.

[7] DETTMEERS T, PAGNONI A, HOLTZMAN A, et al. QLoRA: Efficient finetuning of quantized LLMs[C]//Advances in Neural Information Processing Systems. 2023, 36: 22509-22539.

[8] BAO H, DONG L, PORDING F. BEiT: BERT pre-training of image transformers[C]//International Conference on Learning Representations. 2022.

收稿日期: 2026-06-16

学术指导: 陈志斌 河北赛克普泰计算机咨询服务有限公司

基金项目: 本研究为独立研究工作

作者简介: 贾大林（1986—），男，河北石家庄 独立研究者，主要从事大语言模型推理优化和神经网络架构设计研究。